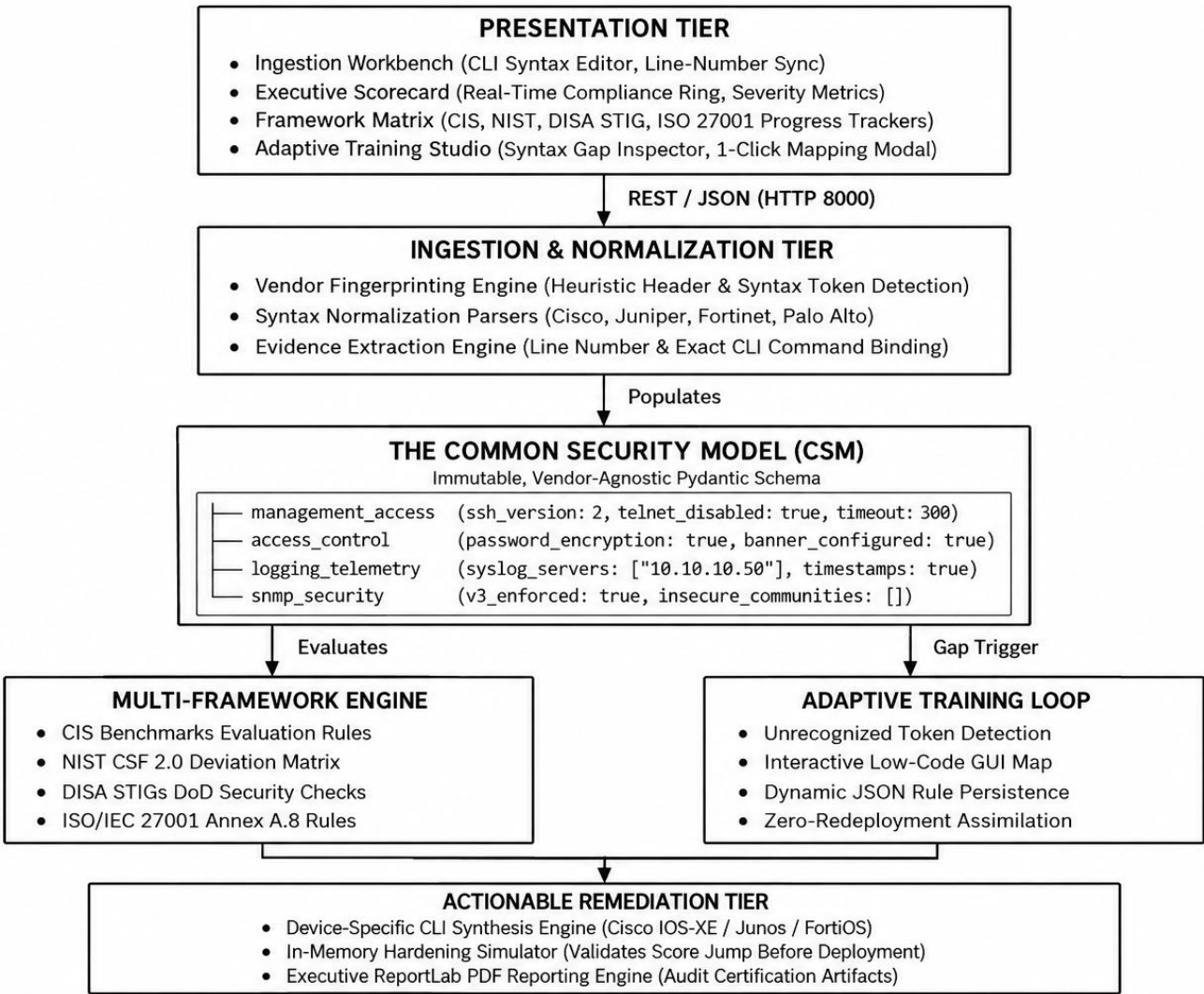


ARCHITECTURE SPECIFICATION DOCUMENT

EliteCore Autonomous Network Security Compliance Auditor

1. Multi-Tier System Architecture



2. Architecture Overview

The **EliteCore Autonomous Network Security Compliance Auditor** is designed to automate the process of checking network device configurations against different security and compliance standards. The architecture follows a simple flow, starting from configuration input and ending with security recommendations and reports.

The **Presentation Tier** is the user-facing layer. It provides tools such as the **Ingestion Workbench**, **Executive Scorecard**, **Framework Matrix**, and **Adaptive Training Studio**.

Through these components, users can upload or inspect device configurations, view compliance scores, check different security frameworks, and understand configuration gaps.

The input is then sent through **REST/JSON APIs** to the **Ingestion & Normalization Tier**. This layer identifies the device vendor, understands its configuration syntax, and extracts important evidence such as exact CLI commands and line numbers. It supports vendors such as **Cisco, Juniper, Fortinet, and Palo Alto**.

The normalized information is stored in the **Common Security Model (CSM)**. This is the core of the system. It provides a common, vendor-independent structure for representing security settings such as management access, access control, logging, and SNMP security. Because all vendor configurations are converted into the same model, the system can evaluate them consistently.

From the CSM, the configuration is processed by two major components. The **Multi-Framework Engine** checks the configuration against standards such as **CIS, NIST CSF, DISA STIGs, and ISO/IEC 27001**. At the same time, the **Adaptive Training Loop** detects unknown or unsupported configuration patterns and helps map them to appropriate security rules without requiring the entire system to be redeployed.

Finally, the results move to the **Actionable Remediation Tier**. This layer generates device-specific CLI commands, uses an in-memory simulator to verify the expected security improvement before deployment, and produces **PDF-based audit and certification reports**.

Overall, the architecture creates a complete automated pipeline:

Configuration Input → Vendor Detection → Normalization → Common Security Model → Multi-Framework Evaluation → Gap Detection → Remediation → Audit Report

This approach reduces manual auditing effort, supports multiple vendors and security frameworks, and makes the compliance process more consistent, traceable, and easier to manage.